

# 改进SKILL agent

不再让主 agent 自己决定是否读 skill，尝试工程化但保留模型语义判断能力，所以定义一个 router-skill-agent 作为 skill 匹配判断的中间件。

HumanMessage("帮我查一下北京现在的天气")

↓

before\_model middleware 触发

↓

router\_skill\_agent 判断：

```
use_skill=True  
skill_path="get_weather/SKILL.md"
```

↓

代码调用 read\_file.invoke(...)

↓

读取 SKILL.md 内容

↓

before\_model 返回新的 messages 更新

↓

主 model 收到：

```
用户问题  
skill 内容
```

↓

主 model 决定调用 search

↓

ToolMessage(search result)

↓

## 主 model 最终回答

```
In [ ]: """
用户提问
↓
before_model 运行 router-skill-agent
↓
router 判断是否匹配 skill
↓
如果匹配，代码硬控制 read_file
↓
把 SKILL.md 内容注入 messages
↓
主 agent 基于 skill 内容执行任务
"""
```

```
In [ ]: import os
from typing import Any

from pydantic import BaseModel, Field

from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import before_model
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, SystemMessage

from langchain.messages import RemoveMessage
from langgraph.graph.message import REMOVE_ALL_MESSAGES
from langgraph.runtime import Runtime

from langchain_community.tools.file_management import ReadFileTool
from langchain_tavily import TavilySearch
from langchain_deepseek import ChatDeepSeek

# =====
# 1. 初始化模型
# =====

model = ChatDeepSeek(
    model="deepseek-chat",
    api_key="",
    base_url="https://api.deepseek.com"
)

# =====
# 2. 初始化底层文件读取能力
# 只允许读取 ./skills 目录下的文件
# =====

raw_read_file = ReadFileTool(root_dir="./skills")

@tool
```

```

def read_file(file_path: str) -> str:
    """
    用于读取 Agent Skills 的定义文件 SKILL.md。

    注意:
    - file_path 必须是相对于 ./skills 的路径
    - 例如: 'get_weather/SKILL.md'
    """
    try:
        return raw_read_file.invoke({"file_path": file_path})
    except Exception as e:
        return f"Error reading file: {str(e)}"

# =====
# 3. 初始化普通联网搜索工具
# 这个工具给主 Agent 使用
# =====

os.environ["TAVILY_API_KEY"] = ""

raw_search = TavilySearch(max_results=2)

@tool
def general_search(query: str) -> str:
    """
    普通联网搜索工具。

    使用场景:
    - 用户问题不匹配任何 skill, 但仍然需要联网查询;
    - 或者某个 skill 已经被系统注入上下文后,
      主 Agent 根据 skill 步骤执行联网搜索。
    """
    return str(raw_search.invoke(query))

# =====
# 4. 扫描 skills 目录, 生成 Skills Snapshot
# =====

def generate_skills_snapshot(skills_dir: str = "./skills") -> str:
    """
    扫描 skills 文件夹, 生成一个简单的 XML 风格技能快照。

    约定目录结构示例:
    skills/
        get_weather/
            SKILL.md
        summarize_openclaw/
            SKILL.md

    返回示例:
    <skills>
        <skill>
            <name>get_weather</name>
    """

```

```

        <path>get_weather/SKILL.md</path>
    </skill>
</skills>
"""

skill_items = []

if not os.path.exists(skills_dir):
    return "<skills></skills>"

for skill_name in os.listdir(skills_dir):
    skill_path = os.path.join(skills_dir, skill_name)
    skill_md_path = os.path.join(skill_path, "SKILL.md")

    if os.path.isdir(skill_path) and os.path.exists(skill_md_path):
        relative_path = f"{skill_name}/SKILL.md"

        skill_items.append(
            f"""
<skill>
    <name>{skill_name}</name>
    <path>{relative_path}</path>
</skill>
            """
        )

snapshot = "<skills>\n" + "\n".join(skill_items) + "\n</skills>"
return snapshot

snapshot_content = generate_skills_snapshot()

# =====
# 5. 定义 Router Agent 的结构化输出
# =====

class SkillRouteResult(BaseModel):
    """
    Router Agent 的判断结果。

    use_skill:
        是否需要使用某个 skill。

    skill_path:
        如果需要使用 skill，则返回对应 SKILL.md 路径。
        必须来自 SKILLS_SNAPSHOT 中的 <path>。

    reason:
        简要说明为什么匹配或不匹配。
    """

    use_skill: bool = Field(
        ...,
        description="用户请求是否匹配某个 Agent Skill"
    )

```

```

skill_path: str | None = Field(
    default=None,
    description="匹配到的 SKILL.md 路径; 如果不匹配任何 skill, 则为 None"
)

reason: str = Field(
    ...,
    description="路由判断原因"
)

# =====
# 6. 创建 Router Agent
# 注意: Router Agent 只负责判断, 不执行任务, 不调用工具
# =====

router_system_prompt = f"""
你是一个 Agent Skill Router。

你的任务:
只判断用户请求是否匹配下面的技能列表, 不要回答用户问题, 不要执行任务。

【SKILLS_SNAPSHOT】
{snapshot_content}

【判断规则】
1. 如果用户请求明显适合某个 skill, 返回 use_skill=True。
2. 如果 use_skill=True, skill_path 必须填写该 skill 的 <path>。
3. 如果用户请求不匹配任何 skill, 返回 use_skill=False, skill_path=None。
4. 你只做路由判断, 不要调用工具, 不要回答用户问题。
"""

router_agent = create_agent(
    model=model,
    system_prompt=router_system_prompt,
    response_format=SkillRouteResult,
)

# =====
# 7. 工具函数: 获取最近一条用户消息
# =====

def get_last_user_message(messages) -> str:
    """
    从 AgentState["messages"] 中倒序查找最近一条 HumanMessage。
    """
    for msg in reversed(messages):
        if isinstance(msg, HumanMessage) and isinstance(msg.content, str):
            return msg.content
    return ""

# =====
# 8. 工具函数: 检查当前 messages 中是否已经注入过某个 skill

```

```
# =====

def has_skill_context(messages, skill_path: str) -> bool:
    """
    防止在多轮 tool loop 中重复注入同一个 SKILL.md。

    我们通过固定 marker 判断上下文里是否已经有该 skill。
    """
    marker = f"[SKILL_PATH]: {skill_path}"

    for msg in messages:
        if isinstance(msg, SystemMessage) and isinstance(msg.content, str):
            if marker in msg.content:
                return True

    return False

# =====
# 9. before_model 中间件:
#     在主模型调用前, 先运行 router agent;
#     如果命中 skill, 由代码强制读取 SKILL.md;
#     然后把 skill 内容注入主 Agent 的 messages。
# =====

@before_model
def inject_skill_context(
    state: AgentState,
    runtime: Runtime
) -> dict[str, Any] | None:
    """
    Skill 注入中间件。

    执行时机:
    - 主 Agent 每次调用模型前执行。

    核心逻辑:
    1. 获取最近用户问题。
    2. 调用 router_agent 判断是否匹配 skill。
    3. 如果不匹配, 什么都不做。
    4. 如果匹配, 代码强制读取对应 SKILL.md。
    5. 将 SKILL.md 内容作为 SystemMessage 注入主 Agent 上下文。
    """

    messages = state["messages"]
    user_query = get_last_user_message(messages)

    if not user_query:
        return None

    # -----
    # Step 1: 调用 router agent 做语义路由判断
    # -----

    route_response = router_agent.invoke(
        {
```

```

        "messages": [
            {
                "role": "user",
                "content": user_query
            }
        ]
    }
)

route_result = route_response["structured_response"]

# -----
# Step 2: 如果不需要 skill, 则不修改 state
# -----

if not route_result.use_skill:
    print("[Skill Router] 未匹配任何 skill, 走普通 Agent 流程。")
    return None

skill_path = route_result.skill_path

if not skill_path:
    print("[Skill Router] 判断需要 skill, 但未返回 skill_path, 跳过注入。")
    return None

# -----
# Step 3: 避免重复注入同一个 skill
# -----

if has_skill_context(messages, skill_path):
    print(f"[Skill Router] skill 已注入过: {skill_path}, 跳过重复注入。")
    return None

# -----
# Step 4: 代码硬控制读取 SKILL.md
# 注意: 这里不是让主 Agent 自己调用 read_file,
#       而是 middleware 层直接调用 read_file。
# -----

print(f"[Skill Router] 命中 skill: {skill_path}")
print(f"[Skill Router] 路由原因: {route_result.reason}")
print("[Skill Router] 正在读取 SKILL.md...")

skill_content = read_file.invoke(
    {
        "file_path": skill_path
    }
)

# -----
# Step 5: 构造注入消息
# -----

skill_message = SystemMessage(
    content=f"""
【已加载 Agent Skill】

```

```
[SKILL_PATH]: {skill_path}
```

```
[ROUTER_REASON]:
{route_result.reason}
```

```
[SKILL_CONTENT]:
{skill_content}
```

【执行要求】

- 当前用户问题已被系统判定需要使用上述 skill。
- 你必须严格按照 [SKILL\_CONTENT] 中的步骤执行任务。
- 不要忽略该 skill。
- 如果 skill 中要求使用联网搜索，请使用 general\_search 工具。
- 如果 skill 信息不足以完成任务，你可以向用户澄清。

```
"""
```

```
)
```

```
# -----
# Step 6: 将 skill_message 插入 messages 最前面
#
# 为什么不直接 return {"messages": [skill_message]}?
# 因为那通常是“追加消息”。
#
# 这里我们希望 skill 上下文出现在用户消息之前，
# 所以采用：
#   先清空 messages
#   再重建 [skill_message, *messages]
# -----
```

```
return {
    "messages": [
        RemoveMessage(id=REMOVE_ALL_MESSAGES),
        skill_message,
        *messages,
    ]
}
```

```
# =====
# 10. 主 Agent 的 system prompt
# =====
```

```
main_system_prompt = f"""
你是一个基于 Agent Skills 架构的智能助手。
```

```
【SKILLS_SNAPSHOT】
{snapshot_content}
```

【工作规则】

1. 如果上下文中存在【已加载 Agent Skill】，说明系统已经为你完成了 skill 路由和 SKILL.md 读取。
2. 你必须严格按照已加载的 SKILL.md 内容执行任务。
3. 如果没有【已加载 Agent Skill】，说明用户问题未匹配任何 skill，你可以按普通助手方式回答。
4. 如果需要联网搜索，请使用 general\_search 工具。
5. 不要自己假装读取过 SKILL.md。
6. 不要编造不存在的 skill 内容。



```

"""

# =====
# 11. 创建主 Agent
#
# 注意:
# - 主 Agent 不再暴露 read_file。
# - read_file 只由 before_model middleware 使用。
# - 主 Agent 只拿到 general_search, 用于执行任务。
# =====

core_tools = [
    general_search,
]

agent_executor = create_agent(
    model=model,
    tools=core_tools,
    system_prompt=main_system_prompt,
    middleware=[inject_skill_context],
)

# =====
# 12. 测试运行
# =====

query = "帮我查一下北京现在的天气"

response = agent_executor.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": query
            }
        ]
    }
)

# =====

for i, msg in enumerate(response["messages"]):
    print(f"\n===== Message {i} =====")
    print(type(msg))
    print("name:", getattr(msg, "name", None))
    print("content:", getattr(msg, "content", None))
    print("tool_calls:", getattr(msg, "tool_calls", None))

```